

DNS JS Exec - MITM Proxy Guide

This guide will help you make a DNS server for free, allowing you to run JS on any site using your DNS server.

Prerequisites:

- Chromebook with **Linux development environment** enabled
- Chromebook should be in **Developer Mode** (Downgrade and do Sh1ttyExec)

Instructions:

- **Part 1: Install MITM Proxy** (Free & Open Source)

MITM Proxy is a free, open-source HTTPS proxy that allows you to intercept, inspect, and modify traffic.

Installation:

```
# macOS (using Homebrew)
brew install mitmproxy
```

```
# Ubuntu/Debian
sudo apt update
sudo apt install mitmproxy
```

```
# Python (cross-platform)
pip install mitmproxy
```

```
# Or download pre-built binaries from mitmproxy.org
```

Basic Configuration:

Create a config file ~/.mitmproxy/config.yaml:

```
# Listen on all interfaces
listen_host: 0.0.0.0
listen_port: 8080
```

```
# SSL/TLS settings
ssl_insecure: true
```

```
# Web interface
```

```
web_host: 0.0.0.0
```

```
web_port: 8081
```

```
# Show human-readable output
```

```
console_eventlog_verbosity: info
```

- **Part 2: Set Up dnsmasq for DNS**

dnsmasq is a lightweight DNS forwarder and DHCP server.

Installation:

```
# macOS
```

```
brew install dnsmasq
```

```
# Ubuntu/Debian
```

```
sudo apt install dnsmasq
```

```
# Arch Linux
```

```
sudo pacman -S dnsmasq
```

Configuration:

Edit /etc/dnsmasq.conf (Linux) or /usr/local/etc/dnsmasq.conf (macOS):

```
# Listen on specific interface (or all)
```

```
interface=eth0
```

```
# OR listen on all
```

```
listen-address=127.0.0.1,192.168.1.1
```

```
# Don't forward non-routed addresses
```

```
bogus-priv
```

```
# Custom domain mappings (point to your MITM proxy)
```

```
address=/test.local/192.168.1.100
```

```
address=/intercept.local/192.168.1.100
```

```
# Upstream DNS servers
```

```
server=8.8.8.8
```

```
server=8.8.4.4
```

```
# Cache size
```

```
cache-size=1000
```

```
# Log queries (useful for debugging)
```

```
log-queries
```

```
log-facility=/var/log/dnsmasq.log
```

Start dnsmasq:

```
# Linux
```

```
sudo systemctl enable dnsmasq
```

```
sudo systemctl start dnsmasq
```

```
# macOS
```

```
sudo brew services start dnsmasq
```

```
# Test it
```

```
dig @127.0.0.1 test.local
```

- Part 3: Configure System to Use Your DNS + Proxy

Option A: System-wide DNS

```
# Linux - edit /etc/resolv.conf
```

```
nameserver 127.0.0.1
```

```
# Or use systemd-resolved
```

```
sudo resolvectl dns eth0 127.0.0.1
```

Option B: Router-level DNS

Configure your router's DHCP to hand out your dnsmasq server as the DNS server for all clients.

Configure Proxy on Client Devices:

```
# Export proxy environment variables
```

```
export http_proxy=http://192.168.1.100:8080
```

```
export https_proxy=http://192.168.1.100:8080
```

```
# Or configure system proxy settings
```

```
# macOS: Network Preferences → Advanced → Proxies
```

```
# Windows: Settings → Network → Proxy
```

- Part 4: JavaScript Injection with MITM Proxy

Create a Python script `inject_script.py`:

```
from mitmproxy import http
```

```
from mitmproxy import ctx
```

```
# JavaScript to inject
```

```
INJECTED_JS = """
```

```
<script>
```

```
(function() {
```

```
    console.log("[MITM] Injected script running!");
```

```
    // Your custom JavaScript here
```

```
    window.addEventListener('load', function() {
```

```
        // Example: Add a banner
```

```
        const banner = document.createElement('div');
```

```
        banner.style.cssText =
```

```
'position:fixed;top:0;left:0;right:0;background:#ff6600;color:white;padding:10px;z-index:9
```

```
9999;text-align:center;font-family:sans-serif;';
```

```
        banner.textContent = ' ⚠ Traffic intercepted by MITM Proxy';
```

```
        document.body.appendChild(banner);
```

```
        // Example: Log all fetch requests
```

```
        const originalFetch = window.fetch;
```

```
        window.fetch = function(...args) {
```

```
            console.log('[MITM] Fetch intercepted:', args[0]);
```

```
            return originalFetch.apply(this, args);
```

```
        };
```

```
        // Example: Modify localStorage
```

```
        const originalSetItem = localStorage.setItem;
```

```
        localStorage.setItem = function(key, value) {
```

```
            console.log('[MITM] localStorage.setItem:', key, value);
```

```
            return originalSetItem.call(this, key, value);
```

```
        };
```

```
    });
```

```
}());
```

```
</script>
```

```
"""
```

```

class Injector:
    def response(self, flow: http.HTTPFlow) -> None:
        # Only process HTML responses
        content_type = flow.response.headers.get("content-type", "")

        if "text/html" in content_type:
            # Decode response body
            html = flow.response.content.decode("utf-8", errors="ignore")

            # Inject before closing </head> or </body>
            if "</head>" in html:
                html = html.replace("</head>", f"{INJECTED_JS}</head>", 1)
            elif "</body>" in html:
                html = html.replace("</body>", f"{INJECTED_JS}</body>", 1)
            else:
                # Fallback: prepend to document
                html = INJECTED_JS + html

            # Update response
            flow.response.content = html.encode("utf-8")

            # Update content length
            flow.response.headers["content-length"] = str(len(flow.response.content))

            ctx.log.info(f"[+] Injected JS into: {flow.request.pretty_url}")

addons = [Injector()]

```

Run MITM Proxy with the script:

```

# Run with injection script
mitmproxy -s inject_script.py

```

```

# OR run web interface
mitmweb -s inject_script.py

```

- **Part 5: Advanced Injection Techniques**

Target Specific Domains:

```

TARGET_DOMAINS = ["example.com", "target.local"]

```

```
def response(self, flow: http.HTTPFlow) -> None:
    host = flow.request.pretty_host

    if any(domain in host for domain in TARGET_DOMAINS):
        # Only inject for target domains
        # ... injection code ...
```

External JavaScript File:

```
import os
```

```
JS_FILE = "/path/to/custom.js"
```

```
def load(self, loader):
    loader.add_option(
        "jsfile", str, JS_FILE,
        "Path to JavaScript file to inject"
    )
```

```
def response(self, flow: http.HTTPFlow) -> None:
    if os.path.exists(ctx.options.jsfile):
        with open(ctx.options.jsfile, 'r') as f:
            js_code = f"<script>{f.read()}</script>"
            # ... inject js_code ...
```

Modify JSON/API Responses:

```
def response(self, flow: http.HTTPFlow) -> None:
    if "application/json" in flow.response.headers.get("content-type", ""):
        import json
        data = json.loads(flow.response.content)

        # Modify data
        data["injected"] = True
        data["mitm"] = "proxy"

        flow.response.content = json.dumps(data).encode()
```

- **Part 6: Complete Setup Script**

Save as setup_mitm.sh:

```
#!/bin/bash
```

```
# Install dependencies
echo "[*] Installing dependencies..."
if [[ "$OSTYPE" == "linux-gnu"* ]]; then
    sudo apt update
    sudo apt install -y mitmproxy dnsmasq
elif [[ "$OSTYPE" == "darwin"* ]]; then
    brew install mitmproxy dnsmasq
fi
```

```
# Create injection script
cat > inject.py << 'EOF'
from mitmproxy import http, ctx
```

```
JS_PAYLOAD = """
<script>
(function() {
    console.log("%c[MITM Proxy] Injected!", "color: #ff6600; font-size: 14px;");

    // Intercept XHR
    const open = XMLHttpRequest.prototype.open;
    XMLHttpRequest.prototype.open = function(method, url) {
        console.log("[MITM] XHR:", method, url);
        return open.call(this, method, url);
    };
})();
</script>
"""
```

```
class JSInjector:
    def response(self, flow: http.HTTPFlow) -> None:
        if "text/html" in flow.response.headers.get("content-type", ""):
            html = flow.response.content.decode("utf-8", errors="ignore")

            if "</head>" in html:
                html = html.replace("</head>", JS_PAYLOAD + "</head>", 1)
                flow.response.content = html.encode("utf-8")
                flow.response.headers["content-length"] = str(len(flow.response.content))
                ctx.log.info(f"[+] Injected: {flow.request.pretty_url}")

addons = [JSInjector()]
```

EOF

```
# Create dnsmasq config
cat > dnsmasq.conf << 'EOF'
listen-address=127.0.0.1
port=53
bogus-priv
cache-size=1000
server=8.8.8.8
log-queries
EOF
```

```
echo "[*] Setup complete!"
echo "[*] Run: sudo dnsmasq -C dnsmasq.conf"
echo "[*] Run: mitmweb -s inject.py --web-host 0.0.0.0"
```

- **Part 7: Testing & Verification**

1. Test DNS:

```
# Query your dnsmasq
dig @127.0.0.1 google.com
```

```
# Check logs
tail -f /var/log/dnsmasq.log
```

2. Install MITM Certificate:

```
# Export certificate
mitmproxy --dump-ca-cert

# Install on client devices
# macOS: sudo security add-trusted-cert -d -r trustRoot -k
/Library/Keychains/System.keychain ~/.mitmproxy/mitmproxy-ca-cert.pem
# Linux: sudo cp ~/.mitmproxy/mitmproxy-ca-cert.pem
/usr/local/share/ca-certificates/mitmproxy.crt && sudo update-ca-certificates
```

- **Credits**

MITM Proxy:

Official site: <https://mitmproxy.org>

Docs: <https://docs.mitmproxy.org>

GitHub: <https://github.com/mitmproxy/mitmproxy>

dnsmasq:

Official site: <https://thekelleys.org.uk/dnsmasq/doc.html>

Man page: <https://thekelleys.org.uk/dnsmasq/docs/dnsmasq-man.html>

Related concepts:

Python mitmproxy scripting API: <https://docs.mitmproxy.org/stable/addons-scripting/>

Guide Maker:

2pro12342 | Github